# C++ STL 算法竞赛基础训练题（10道）

## 第1题：向量排序（vector + sort）

### 题目描述

给定 `n` 个整数，请你将它们按照从小到大的顺序排序后输出。

### 输入格式

第一行包含一个正整数 `n` （$1 \le n \le 10^5$），表示整数的个数。

第二行包含 `n` 个整数，每个整数的绝对值不超过 $10^9$，整数之间用单个空格分隔。

### 输出格式

输出一行，包含 `n` 个整数，表示排序后的结果，整数之间用单个空格分隔，行末可以有多余的空格。

### 样例输入1

Code block

```
5
3 1 4 1 5
```

### 样例输出1

Code block

```
1 1 3 4 5
```

### 样例输入2

Code block

```
6
-5 0 7 -3 0 2
```

### 样例输出2

Code block

```
-5 -3 0 0 2 7
```

### 考察点

1. **vector 容器的基本使用**：定义 `vector<int>` 对象、使用 `push_back()` 函数向 vector 中添加元素。
2. **sort() 函数的应用**：`sort()` 位于 `<algorithm>` 头文件中，默认对区间 `[first, last)` 内的元素进行升序排序，时间复杂度为 $O(n \log n)$，是处理 $10^5$ 级别数据的核心工具。
3. **输入输出效率优化**：由于 `n` 可能达到 $10^5$，建议使用 `scanf()` / `printf()`，或在使用 `cin` / `cout` 前添加 `ios::sync_with_stdio(false); cin.tie(0);` 关闭同步以提升速度。

### 关键提示

- 必须包含头文件：`#include <vector>`（用于 vector）和 `#include <algorithm>`（用于 sort）。
- 对 vector 排序的标准写法：`sort(vec.begin(), vec.end())`。
- **严禁使用冒泡排序等 $O(n^2)$ 算法**，否则对于 $n = 10^5$ 的数据会直接超时。

## 第2题：字符串处理（string）

### 题目描述

给定一个仅由大写字母、小写字母和数字组成的字符串，请你分别统计其中大写字母、小写字母和数字的个数。

### 输入格式

输入一行，包含一个字符串 `s`，字符串的长度不超过 $10^5$，且仅包含大写字母（`'A'-'Z'`）、小写字母（`'a'-'z'`）和数字（`'0'-'9'`）。

### 输出格式

输出三行：

第一行一个整数，表示大写字母的个数；

第二行一个整数，表示小写字母的个数；

第三行一个整数，表示数字的个数。

## 样例输入1

Code block

```
Hello123World
```

## 样例输出1

Code block

```
2
8
3
```

## 样例输入2

Code block

```
2024ACM
```

## 样例输出2

Code block

```
3
0
4
```

## 考察点

1. **string 类的基本操作**：使用 `size()` 或 `length()` 获取字符串长度。
2. **字符串的遍历**：支持下标访问（`s[i]`）和范围 for 循环（`for (char c : s)`）两种方式。
3. **字符判断函数**：使用 `isupper()`（判断大写）、`islower()`（判断小写）、`isdigit()`（判断数字），这些函数位于 `<cctype>` 头文件中。

## 关键提示

- **字符判断的安全性**：上述函数的参数要求为 `unsigned char` 或 `EOF`，否则对于负数字符会导致未定义行为。建议使用 `isupper(static_cast<unsigned char>(c))` 的形式。
- 务必包含 `<cctype>` 头文件，否则字符判断函数可能无法正常工作。

- 初始化三个计数器为 0，遍历字符串时根据字符类型递增对应计数器即可。

## 第3题：括号匹配（stack）

### 题目描述

给定一个仅由字符 `'('` 和 `')'` 组成的字符串，请你判断该字符串中的括号是否匹配。

括号匹配的定义是：每个左括号 `'('` 都有且仅有一个对应的右括号 `')'`，且任意一个右括号 `')'` 都有一个在它之前的左括号 `'('` 与之对应。

### 输入格式

输入一行，包含一个字符串 `s`，字符串的长度不超过 $10^5$，且仅由 `'('` 和 `')'` 组成。

### 输出格式

如果括号匹配，输出一行 `Yes`（不包含引号）；否则输出一行 `No`（不包含引号）。

### 样例输入1

Code block

```
(())()
```

### 样例输出1

Code block

```
Yes
```

### 样例输入2

Code block

```
())
```

### 样例输出2

Code block

```
No
```

### 样例输入3

```
Code block
1   ((())
```

### 样例输出3

```
Code block
1   No
```

### 考察点

1. **stack 容器的核心操作**：`push()`（压入栈顶）、`pop()`（弹出栈顶）、`top()`（获取栈顶元素）、`empty()`（判断栈是否为空），stack 位于 `<stack>` 头文件中。
2. **栈的“后进先出”特性**：这是解决括号匹配问题的关键，利用栈来跟踪未闭合的左括号。

### 关键提示

- 算法流程：
  a. 初始化一个空栈 `stack<char> stk`。
  b. 遍历字符串中的每个字符：
     - 若遇到 `'('`，将其压入栈；
     - 若遇到 `')'`，先检查栈是否为空——若为空，直接返回 `No`；否则弹出栈顶的 `'('`。
  c. 遍历结束后，必须再次检查栈是否为空——若不为空，说明有多余的左括号，返回 `No`；否则返回 `Yes`。
- 注意：栈为空时遇到右括号是最常见的错误情况，必须优先判断。

## 第4题：队列模拟（queue）

### 题目描述

有 `n` 个人编号为 `1` 到 `n`，按照编号从小到大的顺序排成一队。现在进行如下操作：

每次从队首出列一个人，然后将当前队首的人移到队尾，重复这个过程直到所有人都出列。

请你按照出列的顺序输出这 `n` 个人的编号。

## 输入格式

输入一行，包含一个正整数 `n` （$1 \le n \le 10^5$），表示人数。

## 输出格式

输出一行，包含 `n` 个整数，按照出列顺序排列，整数之间用单个空格分隔，行末可以有多余的空格。

## 样例输入1

Code block

```
5
```

## 样例输出1

Code block

```
1 3 5 2 4
```

## 样例输入2

Code block

```
6
```

## 样例输出2

Code block

```
1 3 5 2 6 4
```

## 考察点

1. **queue 容器的核心操作**：`push()` （入队尾）、`pop()` （出队首）、`front()` （获取队首元素）、`empty()` （判断队列是否为空），queue 位于 `<queue>` 头文件中。
2. **队列的“先进先出”特性**：完美模拟排队场景，确保操作顺序符合题目要求。

## 关键提示

- 算法流程：
  a. 初始化队列 `queue<int> q`，并将 `1` 到 `n` 依次入队。

b. 循环处理直到队列为空：
    - 第一步：取出队首元素 `q.front()` 并输出，然后 `q.pop()`。
    - 第二步：**若队列不为空**，取出新的队首元素 `q.front()`，先 `q.pop()` 再 `q.push()`（即移到队尾）。

- 注意第二步的“队列不为空”判断——当最后一个人出列后，队列已空，无需再执行移到队尾的操作。
- 队列的所有操作时间复杂度均为 $O(1)$，整体复杂度 $O(n)$，可轻松处理 $10^5$ 数据。

## 第5题：学生成绩排序（pair + vector + sort 自定义比较）

### 题目描述

给定 `n` 个学生的姓名和成绩，请你按照以下规则对学生进行排序：

1. 首先按成绩从高到低排序；
2. 如果成绩相同，则按姓名的字典序从小到大排序。

请输出排序后的学生信息。

### 输入格式

第一行包含一个正整数 `n`（$1 \le n \le 10^5$），表示学生的人数。

接下来 `n` 行，每行包含一个字符串 `name`（不包含空格，长度不超过 20）和一个整数 `score`（$0 \le score \le 1000$），表示学生的姓名和成绩，姓名和成绩之间用单个空格分隔。

### 输出格式

输出 `n` 行，每行包含一个学生的姓名和成绩，按照排序后的顺序排列，姓名和成绩之间用单个空格分隔。

### 样例输入1

Code block

```
5
Alice 90
Bob 85
Charlie 90
David 80
Eve 95
```

## 样例输出1

Code block

```
Eve 95
Alice 90
Charlie 90
Bob 85
David 80
```

## 样例输入2

Code block

```
3
Tom 100
Jerry 100
Spike 99
```

## 样例输出2

Code block

```
Jerry 100
Tom 100
Spike 99
```

## 考察点

1. **数据存储方式**：使用结构体（Struct）存储学生信息（结构体不属于 Class，是算法竞赛的常用工具，符合题目要求），定义如下：

   Code block

   ```
   struct Student {
       string name;
       int score;
   };
   ```

   然后用 `vector<Student>` 存储所有学生。

2. **sort() 的自定义比较规则**：通过自定义比较函数或 Lambda 表达式实现题目要求的排序逻辑。比较函数的格式为 `bool compare(const Student &a, const Student &b)`，返回 `true` 表示 `a` 应排在 `b` 前面。
3. **string 的字典序比较**：`string` 类型的 `<` 运算符默认就是字典序比较，直接使用 `a.name < b.name` 即可。

### 关键提示

- 自定义比较函数的实现：

Code block

```
bool compare(const Student &a, const Student &b) {
    if (a.score != b.score) {
        return a.score > b.score; // 成绩高的排前面
    } else {
        return a.name < b.name;   // 成绩相同，姓名字典序小的排前面
    }
}
```

- 调用 sort 时传入比较函数：`sort(vec.begin(), vec.end(), compare)`。
- 也可以使用 Lambda 表达式简化代码：

Code block

```
sort(vec.begin(), vec.end(), [](const Student &a, const Student &b) {
    if (a.score != b.score) return a.score > b.score;
    return a.name < b.name;
});
```

- 输入输出时注意效率：建议用 `cin` 读取姓名（配合 `ios::sync_with_stdio(false)`），用 `scanf` 读取成绩。

## 第6题：去重与查找（set）

### 题目描述

给定 `n` 个整数，然后进行 `q` 次查询，每次查询一个整数 `x`，判断 `x` 是否在之前给定的 `n` 个整数中出现过。

### 输入格式

第一行包含两个正整数 `n` 和 `q` （$1 \le n, q \le 10^5$），分别表示整数的个数和查询的次数。

第二行包含 `n` 个整数，每个整数的绝对值不超过 $10^9$，整数之间用单个空格分隔。

接下来 `q` 行，每行包含一个整数 `x`，表示查询的数，`x` 的绝对值不超过 $10^9$。

## 输出格式

对于每次查询，输出一行：如果 `x` 出现过，输出 `Yes`（不包含引号）；否则输出 `No`（不包含引号）。

## 样例输入1

Code block

```
5 3
1 2 3 2 1
2
4
3
```

## 样例输出1

Code block

```
Yes
No
Yes
```

## 样例输入2

Code block

```
6 4
-5 0 7 -3 0 2
0
-5
8
7
```

## 样例输出2

Code block

```
Yes
```

```
Yes
No
Yes
```

### 考察点

1. **set 容器的核心特性**：set 位于 `<set>` 头文件中，具有**自动去重**和**元素有序**的特点，插入和查找的时间复杂度均为 $O(\log n)$，是处理大量数据查找的首选。
2. **insert() 函数**：将元素插入 set，自动去重，重复插入不会产生错误。
3. **find() 函数**：在 set 中查找指定元素，返回指向该元素的迭代器；若找不到，则返回 `set.end()`。

### 关键提示

- 算法流程：
  a. 初始化 `set<int> s`，将 `n` 个整数依次 `s.insert(x)`，自动完成去重。
  b. 对于每次查询 `x`，判断 `s.find(x) != s.end()`：若成立，输出 `Yes`；否则输出 `No`。
- 注意：set 的有序性在本题中不需要利用，我们只需要它的“去重”和“高效查找”特性。
- **严禁使用数组或 vector 进行遍历查找**，那样时间复杂度为 $O(n)$，对于 $q = 10^5$ 会超时。

## 第7题：单词计数（map）

### 题目描述

给定一段由空格分隔的单词文本，每个单词仅由小写字母组成。请你统计每个单词出现的次数，并按照单词的字典序输出每个单词及其出现的次数。

### 输入格式

输入一行，包含一段由空格分隔的单词文本，单词总数不超过 $10^5$，每个单词的长度不超过 20，且仅由小写字母（`'a'-'z'`）组成，单词之间用单个空格分隔，行首和行末没有空格。

### 输出格式

输出多行，每行包含一个单词和一个整数，表示该单词及其出现的次数，单词和整数之间用单个空格分隔。输出顺序按照单词的字典序从小到大排列。

### 样例输入1

```
hello world hello cpp world
```

## 样例输出1

Code block

```
cpp 1
hello 2
world 2
```

## 样例输入2

Code block

```
algorithm algorithm data structure structure structure
```

## 样例输出2

Code block

```
algorithm 2
data 1
structure 3
```

## 考察点

1. **map 容器的核心特性**：map 位于 `<map>` 头文件中，是一个**键值对（Key-Value）容器**，其中 Key 唯一，且元素按 Key 的字典序自动排序。插入、查找和访问的时间复杂度均为 $O(\log n)$ 。
2. **operator[] 运算符**：使用 `map[key]` 可以访问键为 `key` 的元素；若 `key` 不存在，则自动插入一个键为 `key` 、值为默认值（对于 `int` 是 0）的元素。因此， `cnt[word]++` 可以直接统计单词出现次数。
3. **map 的遍历**：使用范围 for 循环 `for (auto &p : cnt)` 遍历，其中 `p.first` 是 Key（单词）， `p.second` 是 Value（次数）。

## 关键提示

- 初始化 `map<string, int> cnt` 用于存储单词和次数。
- 读取单词时直接使用 `cin` ，因为 `cin` 默认以空格为分隔符，正好逐个读取单词。
- 遍历 map 时，由于 map 已按 Key 字典序排序，直接输出即可，无需额外排序。

- 题目已说明单词仅由小写字母组成，无需处理大小写转换。

## 第8题：Top K 大的数（priority_queue）

### 题目描述

给定 `n` 个整数，请你找出其中前 `k` 大的数，并按照从大到小的顺序输出这 `k` 个数。

### 输入格式

第一行包含两个正整数 `n` 和 `k` （$1 \le k \le n \le 10^5$），分别表示整数的个数和需要找出的前 `k` 大的数的个数。

第二行包含 `n` 个整数，每个整数的绝对值不超过 $10^9$，整数之间用单个空格分隔。

### 输出格式

输出一行，包含 `k` 个整数，表示前 `k` 大的数，按照从大到小的顺序排列，整数之间用单个空格分隔，行末可以有多余的空格。

### 样例输入1

Code block

```
6 3
3 1 4 1 5 9
```

### 样例输出1

Code block

```
9 5 4
```

### 样例输入2

Code block

```
5 5
10 20 30 40 50
```

### 样例输出2

```
50 40 30 20 10
```

### 考察点

1. **priority_queue（优先队列）的核心特性**：位于 `<queue>` 头文件中，队首元素始终是优先级最高的元素（默认是最大元素，即“大顶堆”）。
2. **大顶堆的构造**：`priority_queue<int>` 是默认大顶堆，队首为最大值。
3. **小顶堆的构造（空间优化）**：若 `n` 很大而 `k` 较小，可使用小顶堆维护前 `k` 大的元素，空间复杂度优化为 $O(k)$。构造方式：`priority_queue<int, vector<int>, greater<int>>`。
4. **核心操作**：`push()`（插入元素）、`top()`（获取队首）、`pop()`（移除队首），时间复杂度均为 $O(\log n)$。

### 关键提示

- **方法一：大顶堆（简单直观）**
  a. 将所有 `n` 个元素插入大顶堆。
  b. 依次取出前 `k` 个元素并输出，即为从大到小的顺序。
  优点：代码简单；缺点：空间复杂度 $O(n)$（但对于 $10^5$ 完全可接受）。
- **方法二：小顶堆（空间优化）**
  a. 遍历 `n` 个元素：
     - 若堆大小 `< k`，直接插入；
     - 否则，若当前元素 `>` 堆顶元素（堆顶是当前前 `k` 大中最小的），则弹出堆顶，插入当前元素。
  b. 遍历结束后，堆中是前 `k` 大的元素，但由于是小顶堆，需取出后逆序输出。
  优点：空间复杂度 $O(k)$；缺点：需逆序，代码稍复杂。
- 建议先掌握方法一，再理解方法二。两种方法的时间复杂度均满足要求。

## 第9题：二分查找次数（lower_bound + upper_bound）

### 题目描述

给定一个长度为 `n` 的有序整数数组（从小到大排列），然后进行 `q` 次查询，每次查询一个整数 `x`，求 `x` 在数组中出现的次数。

### 输入格式

第一行包含两个正整数 `n` 和 `q` （$1 \le n, q \le 10^5$），分别表示数组的长度和查询的次数。

第二行包含 `n` 个整数，按照从小到大的顺序排列，每个整数的绝对值不超过 $10^9$，整数之间用单个空格分隔。

接下来 `q` 行，每行包含一个整数 `x`，表示查询的数，`x` 的绝对值不超过 $10^9$。

## 输出格式

对于每次查询，输出一行，包含一个整数，表示 `x` 在数组中出现的次数。

## 样例输入1

Code block

```
6 3
1 2 2 2 3 5
2
4
5
```

## 样例输出1

Code block

```
3
0
1
```

## 样例输入2

Code block

```
5 4
-5 -3 0 0 2
0
-5
3
-3
```

## 样例输出2

Code block

```
2
```

```
1
0
1
```

### 考察点

1. **lower_bound() 函数**：位于 `<algorithm>` 头文件中，`lower_bound(first, last, x)` 返回指向**第一个大于等于** `x` 的元素的迭代器，时间复杂度 $O(\log n)$。
2. **upper_bound() 函数**：位于 `<algorithm>` 头文件中，`upper_bound(first, last, x)` 返回指向**第一个大于** `x` 的元素的迭代器，时间复杂度 $O(\log n)$。
3. **迭代器的差**：`upper_bound` 返回的迭代器减去 `lower_bound` 返回的迭代器，结果就是 `x` 出现的次数——因为这两个迭代器之间的元素恰好是所有等于 `x` 的元素。

### 关键提示

- **必须确保数组是有序的**（从小到大），否则 `lower_bound` 和 `upper_bound` 的结果是未定义的。
- 对于 `vector<int> vec`，调用方式为：

  Code block

  ```
  auto left = lower_bound(vec.begin(), vec.end(), x);
  auto right = upper_bound(vec.begin(), vec.end(), x);
  cout << (right - left) << endl;
  ```

- 迭代器的差类型为 `difference_type`，可直接赋值给 `int` 输出（因为次数不超过 $10^5$，在 `int` 范围内）。
- 若 `x` 不存在，`left` 和 `right` 相等，差为 0，正好符合要求。
- **严禁使用遍历数组统计次数**，否则 $q = 10^5$ 时会超时。

## 第10题：有序数组操作（vector + lower_bound 综合）

### 题目描述

请你维护一个有序的整数数组（从小到大排列），支持以下四种操作：

1. `insert x`：将整数 `x` 插入数组中，插入后数组仍然保持从小到大的顺序。
2. `delete x`：删除数组中第一个等于 `x` 的元素，如果 `x` 不存在于数组中，则不进行任何操作。

3. `count x`：查询整数 `x` 在数组中出现的次数。
4. `kth k`：查询数组中第 `k` 小的元素（`k` 从 1 开始计数，即 `k=1` 表示最小的元素，`k=2` 表示第二小的元素，以此类推），保证查询时 `k` 不超过数组的当前长度。

## 输入格式

第一行包含一个正整数 `m`（$1 \le m \le 10^5$），表示操作的次数。

接下来 `m` 行，每行表示一个操作，操作格式为上述四种之一，其中 `x` 是一个绝对值不超过 $10^9$ 的整数，`k` 是一个正整数。

## 输出格式

对于每个 `count x` 操作和 `kth k` 操作，输出一行一个整数，表示对应的结果。

## 样例输入1

Code block

```
10
insert 3
insert 1
insert 4
insert 1
insert 5
count 1
kth 3
delete 1
count 1
kth 3
```

## 样例输出1

Code block

```
2
3
1
4
```

## 样例输入2

Code block

```
7
```

```
insert 5
insert 2
insert 5
insert 7
count 5
delete 5
count 5
```

## 样例输出2

Code block

```
2
1
```

## 考察点

1. **vector 的 insert() 函数**：`vec.insert(pos, val)` 在迭代器 `pos` 指向的位置插入 `val`，插入后后面的元素自动后移，时间复杂度 $O(n)$（因可能移动大量元素）。
2. **vector 的 erase() 函数**：`vec.erase(pos)` 删除迭代器 `pos` 指向的元素，删除后后面的元素自动前移，时间复杂度 $O(n)$。
3. **lower_bound() 的综合应用**：
   - 用于找到 `insert` 的位置（第一个大于等于 `x` 的位置，插入后保持有序）；
   - 用于找到 `delete` 的位置（第一个等于 `x` 的位置，需先判断是否存在）。
4. **count 操作**：仍使用 `lower_bound` 和 `upper_bound` 的差计算次数。
5. **kth 操作**：由于数组有序，第 `k` 小的元素就是 `vec[k-1]`（下标从 0 开始），直接访问即可，时间复杂度 $O(1)$。

## 关键提示

- 初始化 `vector<int> vec` 维护有序数组。
- **insert x 操作**：

  Code block

  ```
  auto it = lower_bound(vec.begin(), vec.end(), x);
  vec.insert(it, x);
  ```

- **delete x 操作**：

```
auto it = lower_bound(vec.begin(), vec.end(), x);
if (it != vec.end() && *it == x) {
    vec.erase(it);
}
```

- **count x 操作**：

Code block

```
auto left = lower_bound(vec.begin(), vec.end(), x);
auto right = upper_bound(vec.begin(), vec.end(), x);
cout << (right - left) << endl;
```

- **kth k 操作**：

Code block

```
cout << vec[k-1] << endl;
```

- 注意：虽然 vector 的 insert/erase 在中间位置的时间复杂度为 $O(n)$，对于 $m = 10^5$ 可能超时，但本题目的是练习 STL 基本操作和 lower_bound 的应用。在竞赛实际题目中，若需更高效的插入删除，会使用平衡树等高级数据结构，但作为基础题，用 vector 是合适的。
- 读取操作时，先读取操作类型的字符串（如 `"insert"`），再根据类型读取对应参数。

需要我针对某道题提供详细的代码实现和逐行讲解吗？

（注：文档部分内容可能由 AI 生成）